

2

Understanding the ETL Process and Data Pipelines

With a firm foundation of Python under our belts and a clean development environment established, we can now pivot to talking about the fundamentals of data pipelines.

Within this chapter, we will define what a data pipeline is, as well as take a more in-depth look at the process of building robust pipelines. We will then discuss different approaches, such as the **Extract, Transform, and Load** (ETL) and **Extract, Load, and Transform** (ELT) methodologies, and how they tie into effectively automating data movement.

By the end of this chapter, you will have an established workflow for building data pipelines within your local environment and will have covered the following topics:

- What is a data pipeline?
- Creating robust data pipelines
- What is an ETL pipeline? How do ETL pipelines differ from ELT pipelines?
- Automating ETL pipelines
- Examples of use cases of ETL pipelines

What is a data pipeline?

A data pipeline is a series of tasks, such as transformations, filters, aggregations, and merging multiple sources, before outputting the processed data into some target. In layman's terms, a data pipeline gets data from the "source" to the "target," as depicted in the following diagram:

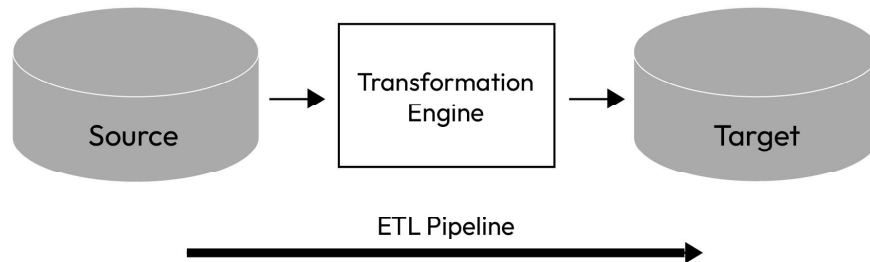


Figure 2.1: A sample ETL process illustration

You can think of pipelines as transport tubes in a mailroom. Mail can be placed in specific tubes and *sucked* up to specific processing centers. Based on specific labels, the mail is then moved and sorted into specific pathways that eventually bring it to its destination. The core concept of data pipelines is quite similar. Like mail, packets of raw data are ingested into the entry of the pipeline and, through a series of steps and processes, the raw material is formatted and packaged into an output location, which is most commonly used for storage.

From a business perspective, the driving incentive for creating data pipelines is to design a system to transform their most valuable asset – raw data – into actionable information (*Reference #2*). Typically, data pipeline architectures feature a unidirectional (one-way) communication flow between the flow of input data sources to output data storage systems (*Reference #1*):

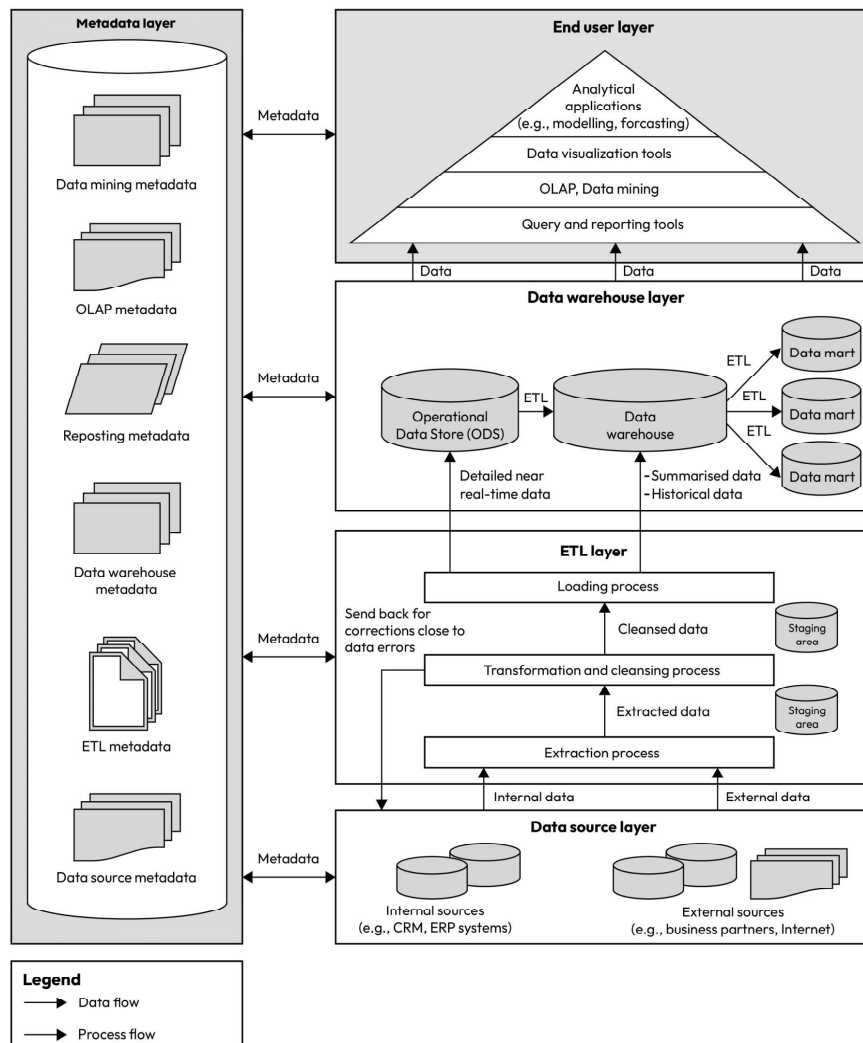


Figure 2.2: Data flow architecture – (Reference #1)

Now that we have a clearer picture of the general structure of a data pipeline, we can dive into what makes a data pipeline *robust* and what doesn't.

How do we create a robust pipeline?

A data pipeline is only as scalable as its foundation is strong. It is crucial to meticulously design an architectural plan, which includes anything from defining the types of data that need to be collected to the methodologies used to analyze the data, to create a sustainable data environment

(*Reference #2*). Just as a data pipeline built with a strong architecture is easily maintainable and scalable, so too is a weak data pipeline at high risk of failure, either structurally or analytically producing an inaccurate product, and having staggering consequences.

The following are the attributes of a robust data pipeline:

- Clearly defined expectations
- Scalable architecture
- Reproducible and clear

A robust data pipeline should have clearly defined expectations in terms of the data it is processing and the results it is expected to produce. This includes specifying the types and sources of data, as well as the desired output format and any required transformations or aggregations. Having clearly defined expectations helps ensure that the pipeline is consistently producing accurate and reliable results.

In addition to clearly defined expectations, a robust data pipeline should have a scalable architecture that can handle increasing volumes of data without degradation in performance. This may involve using distributed systems or implementing efficient algorithms to process the data promptly.

Reproducibility and clarity are also important attributes of a robust data pipeline. The pipeline should be able to produce the same results each time it is run, and the steps involved in processing the data should be documented and easy to understand. This helps ensure that the pipeline can easily be maintained and modified as needed.

Pre-work – understanding your data

Your data is your foundation. It's what dictates the context and flow of the overall data pipeline. As the designer of the pipeline, it is best to become intimately familiar with both the input and output data structure requirements, as follows:

- **Input data:** You should write clear definitions of the input data, such as its data structures, the prevalence of corrupted data, and the frequency of new data creation. What is the likelihood of data corruption, and how can it be accounted for within the pipeline?
- **Output data:** It is important to have a thorough understanding of the structural requirements of the output data to ensure that the project's goals are consistently upheld during its design and construction. A weak pipeline is problematic, but a pipeline that serves no purpose is even worse. It is crucial to achieve the desired result; otherwise, the entire effort becomes pointless.

In summary, it is important to have a clear understanding of both the input and output data structures when designing a data pipeline. This includes knowing the data structures, any potential issues with the data (such as corruption), and the frequency of new data creation for the input data. For the output data, it is important to understand the structural requirements to ensure that the pipeline consistently

produces the desired result. It is also important to ensure that the pipeline serves a purpose and helps achieve the project's goals. If the pipeline does not produce the desired result or serve a purpose, it may be considered pointless.

Design planning – planning your workflow

Before writing a line of code, take some time to sketch out a flowchart of the transformations that need to be performed on the input data to produce the output data. Go a little deeper into how Python packages can be used to accomplish these transformations. This is how you create the foundation for operational stability in your code, where you apply **Don't Repeat Yourself (DRY)** practices to create non-redundant code that is thoroughly tested and easy to logically follow.

DRY practices are a programming principle that aims to reduce redundancy in code. This means avoiding writing the same code multiple times, and instead using functions, variables, and other code structures to reuse code whenever possible.

Architecture development – developing your resources

As the architect responsible for the data pipeline, you need to understand the various types of connections needed to securely link the source(s) of your input data to your development and production environments, as well as the method for connecting to the output location. You will also need to determine the approach for your pipeline and the level of error handling necessary to ensure the long-term stability and reliability of the pipeline.

This is the *meat and potatoes* of the true purpose of a data engineer, and it's a *massive* topic. Don't worry – we will delve more deeply into this topic in *Part 2*.

Putting it all together – project diagrams

Arguably, this is where our methods might diverge a bit from the norm. As strong advocates for *strategic laziness*, getting your entire workflow on paper is an incredibly effective tool for communicating your desired results to both yourself and others. Creating project diagrams is fundamental to a project that is not only clear, reproducible, and fully strategized, but also makes it highly shareable and transparent.

Utilizing visual aids can greatly help with efficiently processing information for most people, making it simple for you to communicate your pipelines to non-technical individuals. This is where you put all of the previous steps together, and add details around each component of the pipeline's workflow to establish what needs to be designed to create the desired output data, in the desired format and storage endpoint.

The following diagram illustrates the end-to-end process of data processing, including the final utilization of the data:

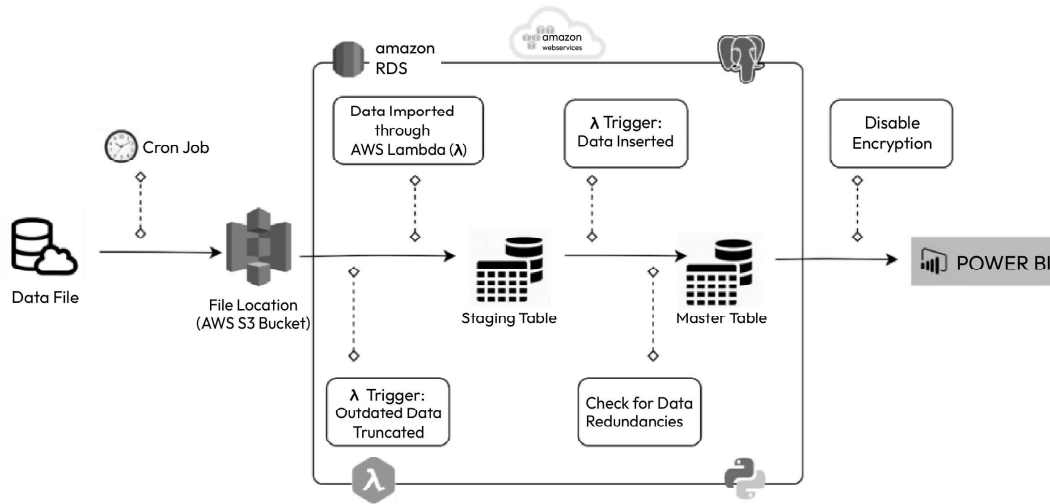


Figure 2.3: Data processing with ETL

Now that we have a deep understanding of the elements that make up a strong data pipeline, let's delve into the various architectural approaches for constructing robust pipelines.

What is an ETL data pipeline?

ETL stands for **Extract, Transform, and Load**. In an ETL process, data is first extracted from a source, then transformed and formatted in a specific way, and finally loaded into a final storage location. These pipelines are useful for organizing and preparing data for future purposes such as performing analysis and model creation smoothly and efficiently:

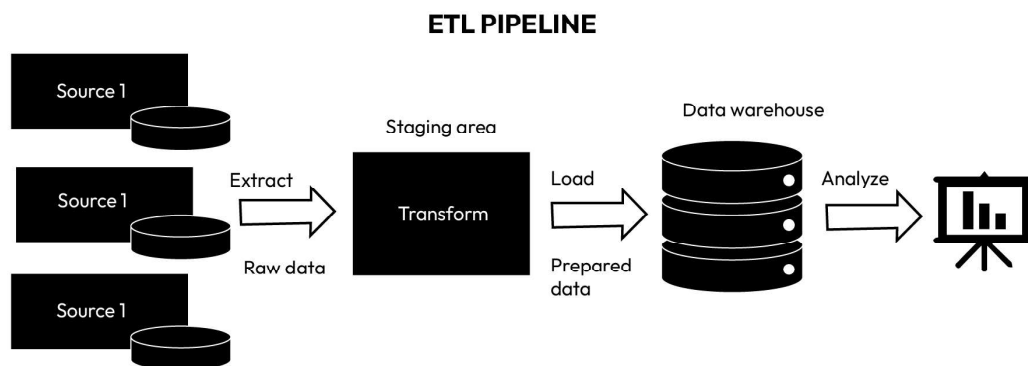


Figure 2.4: Sample ETL pipeline

ELT stands for **Extract, Load, and Transform**, and is similar to ETL, but the data is first loaded into the target system and then transformed within the target system.

Which one to use depends on the specific requirements and characteristics of the systems involved and the data being moved. Here are a few factors that you might consider when deciding between ETL and ELT:

- **Data volume:** If the volume of data is very large, ELT might be more efficient because the transformation step can be done in parallel within the target system
- **Data transformation requirements:** If the data needs to undergo complex transformations, it might be easier to perform the transformations in the target system using ELT
- **Source system capabilities:** If the source system is not able to perform the necessary transformations, ETL might be the only option
- **Target system capabilities:** If the target system is not able to efficiently handle the load phase of the ETL process, ELT might be a better option
- **Data latency:** If real-time data movement is required, ELT might be a better choice because it allows the data to be loaded and transformed more quickly

In general, ETL is more commonly used when the source and target systems are different and the data needs to be transformed in a specific way before it is loaded into the target system. ELT is more commonly used when the target system is more powerful and can handle the transformation step itself:

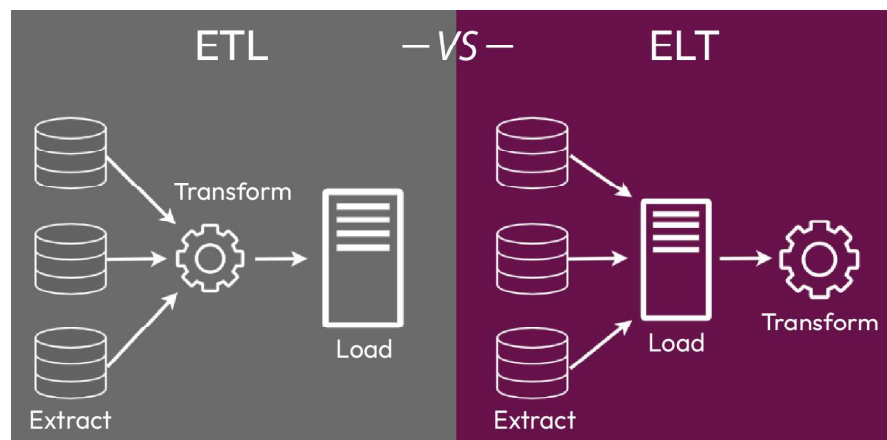


Figure 2.5: ETL versus ELT

In this book, we will focus more exclusively on ETL pipelines, where there are clear output data expectations.

There are three main types of ETL pipelines:

- Batch processing
- Streaming
- Cloud-native

Let's take a look!

Batch processing

Batch processing is a method of data processing that involves dividing a large volume of data into smaller pieces, or batches, and processing each batch separately. This is often done when a project requires a large amount of data to be handled (mostly in TB or above), but the downstream use of the data only requires it to be available asynchronously. Batch processing is a popular choice for handling large amounts of data because it allows engineers to design a process that can run one batch of data through the pipeline at a time, rather than trying to process all of the data at once. This is especially useful for moving large amounts of data at regular intervals.

One example of a batch processing system is a company that processes customer orders for an online store. The company receives a large volume of orders each day and needs to process this data to update inventory levels, generate invoices, and fulfill orders. To do this, they use a batch processing system to break the data into smaller chunks and run each chunk through their data pipeline. This allows them to process the data efficiently, without overwhelming their systems or causing delays in order processing:

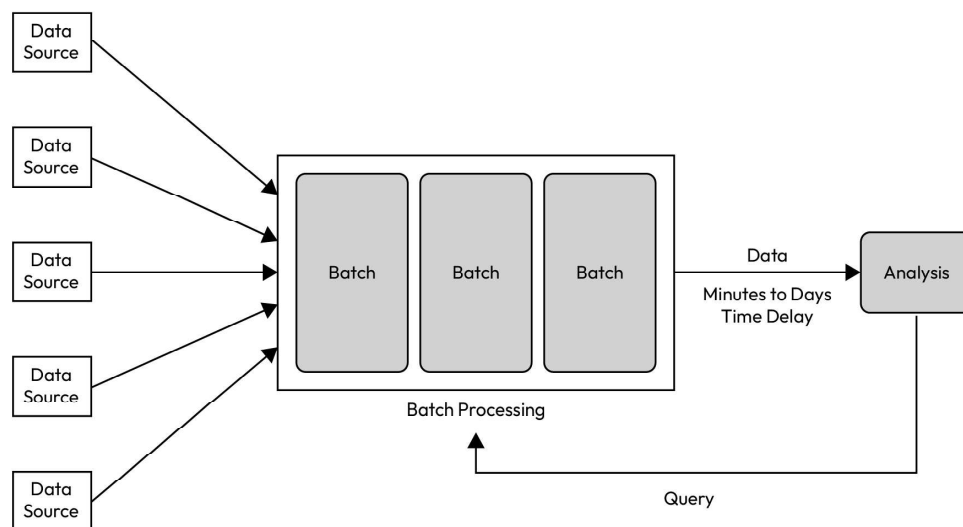


Figure 2.6: An illustration of batch processing

An example of this is ingesting the user's data from an e-commerce website every 30 minutes and passing it through a pipeline to transform and load it into **relational database management system (RDBMS)** tables in a data warehousing setup.

Streaming method

Real-time data solutions are necessary when a project needs to immediately process fresh data. Streaming methods are often used in these situations as they allow data to flow continuously, which may be variable and subject to sudden changes in structure. To handle these challenges, a combination of data pipelines is typically utilized. There are several advanced tools, such as Apache Storm and Apache Samza, that can efficiently process real-time data. We will delve further into this topic in future chapters.

An example of this is an e-commerce website that needs to process real-time user data as shopping is in progress. The use of real-time data processing, combined with AI/ML, can result in an enhanced user shopping experience.

Cloud-native

After the advent of cloud technologies, complex data processing has become very easy. Most of the famous public cloud platforms, such as **Amazon Web Services (AWS)**, **Google Cloud Platform (GCP)**, and **Microsoft Azure**, provide in-built data processing capabilities that come with various integrated tools and technologies that can be used to build a robust and resilient ETL pipeline.

Putting it all together

ETL and ELT are data processing approaches that involve extracting data from a source, performing transformations on it, and loading it into a final storage location. ETL is commonly used when the source and target systems are different and the data needs to be transformed in a specific way before it is loaded into the target system. ELT is more commonly used when the target system is more powerful and can handle the transformation step itself. Factors to consider when deciding between ETL and ELT include data volume, transformation requirements, source and target system capabilities, and data latency. There are three main types of ETL pipelines: batch processing, streaming, and cloud-native. Batch processing is used for large volumes of data that only need to be available asynchronously, streaming is used for real-time data processing, and cloud-native is used for data processing in cloud computing environments. The purpose of this book is to focus on ETL pipelines where there are clear output data expectations.

Next, we will explore the process of automating ETL pipelines.

Automating ETL pipelines

To streamline and optimize the ETL process in a production environment, there are several tools and technologies available to automate the pipeline. These tools are particularly important in an enterprise setting, where the volume and complexity of data can be significant. In this section, we will discuss the most important and relevant tools used in an enterprise environment.

There are several key benefits to automating ETL pipelines:

- **Data democratization:** Automating the ETL process can make it easier for a wider range of users to access and use data since the process of extracting, transforming, and loading data is streamlined and made more efficient
- **Robust data availability and access:** By automating the ETL process, data is made more consistently available and accessible to users since the pipelines are designed to run regularly and can be easily configured to handle any changes or updates to the source data
- **Team focus:** Automating ETL pipelines can free up team members to focus on more important tasks such as data analytics, developing machine learning models, and other higher-value activities, rather than spending time on manual data preparation tasks
- **Onboarding process:** Automated ETL pipelines can make it easier and faster to onboard new team members since the process of extracting and preparing data is already streamlined and automated, reducing the need for new hires to learn complex manual processes
- **Schema management:** Automating ETL pipelines can also help with schema management since the pipelines can be configured to handle changes to the source data schema automatically, ensuring that the data is always properly transformed and loaded into the target system

By utilizing tools such as AWS Lambda and EC2 alongside Step Functions, it is simple to orchestrate or automate ETL pipelines on a cloud platform. For instance, we can implement these resources to automate our pipelines effectively.

Orchestrating ETL pipelines can also be done using open source tools such as Apache Airflow, Luigi, and others (more on this in *Parts 2 and 3*).

Exploring use cases for ETL pipelines

Now, we will cover the benefits and uses of ETL pipelines in organizations:

- Benefits of ETL pipelines:
 - Allow developers and engineers to focus on useful tasks rather than worrying about data
 - Free up time for developers, engineers, and scientists to focus on actual work
 - Help organizations move data from one place to another and transform it into a desired format efficiently and systematically

- Applications of ETL pipelines:
 - Migrating data from a legacy platform to the cloud and vice versa
 - Centralizing data sources to have a consolidated view of data
 - Providing stable data sources for data-driven applications and data analytic tools
 - Acting as a blueprint for organizational data, serving as a single source of truth
- Example of an ETL pipeline in action:
 - Netflix has a very robust ETL pipeline that manages petabytes of data, allowing them to employ a small team of engineers to handle admin tasks related to data
- Overall benefits of ETL pipelines:
 - Saves money in the long run
 - Helps with business expansion

With that, we have come to the end of this chapter.

Summary

In this chapter, we learned about data pipelines and the ETL process, as well as the different approaches and types of ETL pipelines, including batch processing, streaming, and cloud-native. We also learned about the benefits of automating ETL pipelines, such as schema management and data quality. In the next chapter, we will learn about the process of creating a scalable and resilient pipeline.

References

To learn more about the topics that were covered in this chapter, take a look at the following resources:

1. *ETL and its impact on Business Intelligence*: https://www.academia.edu/11434594/ETL_and_its_impact_on_Business_Intelligence?email_work_card=title
2. *A Five-Layered Business Intelligence Architecture*: https://www.academia.edu/25962611/A_Five_Layered_Business_Intelligence_Architecture?email_work_card=view-paper